Movers SAT Solver: Report

Optimal Logistics Planning Using Boolean Satisfiability

Group: Group C

Members:
Giovanni Barbieri
Vasyl Damian
Simona Di Lena
Cristian Nicolae Gheorghe
Simone Maccario
Patrick Magnani
Massimiliano Nicol

Course: Theory of Computation

May 20, 2026

This report describes the design and implementation of a SAT-based solver for the movers problem.

Contents

1 Introduction **4**

2 System Architecture **5**
2.1 Backend Architecture 5
2.2 Frontend Architecture 6

3 Theoretical Background **7**

4 Boolean Variable Representation **8**

5 Constraint Encoding **9**
5.1 Common Cardinality Constraints 9
5.2 Task 1: Movement-Only Encoding 9
5.3 Task 2: Movement and Carrying Encoding 10

6 Solver Execution and Model Decoding **13**
6.1 Fixed-Horizon Solving 13
6.2 Optimal Solving by Iterative Deepening 13
6.3 Model Decoding 13

7 DIMACS and Variable Mapping Output **14**
7.1 DIMACS CNF 14
7.2 Variable Mapping 14

8 Frontend Functionality **15**
8.1 Problem Setup Panel 15
8.2 Results Panel 15
8.3 Building Visualizer 15
8.4 Action Log 16

9 Complexity and Scalability Analysis **17**
9.1 Variable Growth 17
9.2 Clause Growth 17
9.3 SAT Solving Complexity 17
9.4 Practical Scalability 17

10 Experimental Evaluation **18**
10.1 Case 1: Task 1 Movement with Different Goal Floors 18
10.2 Case 2: Task 1 Unsatisfiable Within a Limited Horizon 19
10.3 Case 3: Trivial Task 1 Goal 20
10.4 Case 4: Task 2 with Three Items 20
10.5 Case 5: Task 2 Unsatisfiable Fixed-Horizon Instance 20
10.6 Case 6: Full Example with Five Items 21
10.7 Case 7: Larger Custom Frontend Instance 21

11 Design Decisions and Issues **24**
11.1 General CNF Instead of 3-CNF 24
11.2 Pairwise Cardinality Encoding 24
11.3 Iterative Deepening for Optimality 24
11.4 Readable Mapping Output 24
11.5 Frontend Visualization 24

12 Limitations **25**

13 Conclusion **26**

Appendices **27**

A User Guide **27**
A.1 Docker Setup 27
A.2 Backend Setup Without Docker 27
A.3 Frontend Setup Without Docker 28
A.4 CLI Examples 28

B API Endpoints **29**

C Example API Request **30**

D File Structure **31**

1 Introduction

The goal of this project is to model and solve the *movers problem* using Boolean satisfiability (SAT). The problem consists of emptying an n-floor building with a team of m movers. The building contains a number of items located on different floors, and all persons and items must eventually end on the ground floor, represented as floor 0.
Time is assumed to be discrete. At every time step, each mover may perform one of the available actions:

$$\texttt{ascends}(p,t), \qquad \texttt{descends}(p,t), \qquad \texttt{carries}(p,i,t),$$

where p is a person, i is an item, and t is a time step. The action `ascends` moves a person up one floor, `descends` moves a person down one floor, and `carries` represents a person carrying an item down one floor while descending. Each action takes exactly one time step.
The project was implemented as a complete SAT-based logistics planning system. The backend encodes the problem as a conjunctive normal form (CNF) formula and solves it using the Glucose3 SAT solver through the PySAT library. The frontend provides an interactive graphical interface where users can configure the number of floors, movers, items, and solving mode, then visualize the computed plan step by step.
The final system supports two task variants:

- **Task 1: Movement Only.** Movers start at floor 0 and must reach specified target floors within a given time horizon.
- **Task 2: Movement and Carrying.** Movers and items must all end on floor 0. Items can only move downward when carried by a descending mover.

The main objective of the solver is not only to check whether a plan exists for a fixed number of steps, but also to find the shortest valid plan using iterative deepening over the time horizon.

2 System Architecture

The system is divided into two main components: a Python backend and a Vue frontend. The backend is responsible for the formal SAT encoding, solver execution, model decoding, and output generation. The frontend is responsible for user interaction, input validation, and visualization of the computed plan.

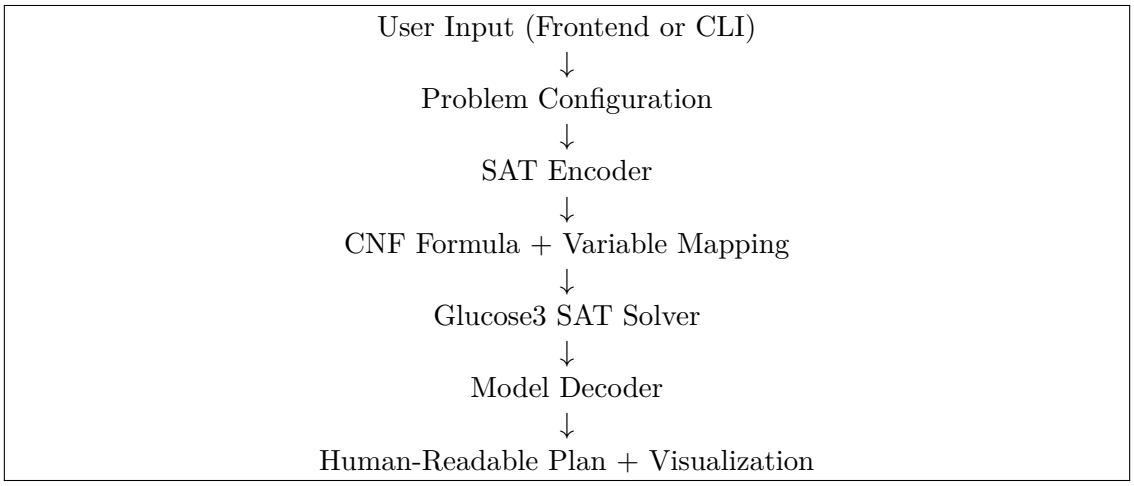


Figure 1: General pipeline of the Movers SAT Solver.

2.1 Backend Architecture

The backend is implemented in Python using FastAPI. It exposes REST endpoints that allow the frontend to send a problem instance and receive either a satisfiable plan or an unsatisfiable result.
The backend is organized into the following main modules:

- `models/problem.py`: defines the Pydantic models for problem inputs and solver outputs.
- `encoding/variables.py`: defines the Boolean variable families and maps symbolic variables to integer identifiers.
- `encoding/common_clauses.py`: provides reusable CNF helpers such as at-most-one, at-least-one, and exactly-one constraints.
- `encoding/task1_encoder.py`: encodes the movement-only problem.
- `encoding/task2_encoder.py`: extends Task 1 with item movement and carrying constraints.
- `solver/runner.py`: runs Glucose3 either for a fixed time horizon or with iterative deepening.
- `solver/decoder.py`: converts a satisfying SAT assignment into a readable sequence of time steps.
- `output/dimacs.py`: serializes the generated CNF formula in DIMACS format.
- `output/mapping.py`: exports the variable mapping as JSON.
- `api/routes.py`: defines the REST API endpoints.

The API provides endpoints for solving a problem optimally, solving at a fixed horizon, downloading the DIMACS CNF file, and downloading the variable mapping.

2.2 Frontend Architecture

The frontend is implemented using Vue and Tailwind CSS. It communicates with the backend through a small API client that sends JSON payloads to the FastAPI server and retrieves solver results.
The frontend is divided into several components:

- `App.vue`: maintains the global form state and coordinates solver requests.
- `ProblemSetup.vue`: provides the form where the user selects the task type, number of floors, number of movers, items, and advanced solver options.
- `ResultsPanel.vue`: displays solver status, runtime, number of steps, playback controls, and download buttons.
- `BuildingVisualizer.vue`: renders the building floors and places movers and items on the correct floors for the selected time step.
- `ActionLog.vue`: lists the actions performed by each mover at the current time step.

The interface allows two solving modes:

- **Iterative Deepening:** searches for the shortest valid solution up to a maximum number of steps.
- **Fixed Horizon:** checks whether a solution exists for an exact number of time steps.

When a solution is found, the user can play through the sequence using timeline controls. The frontend allows the user to download the generated DIMACS CNF instance and the variable mapping. The satisfying model returned by the SAT solver is decoded into the displayed step-by-step plan.

3 Theoretical Background

Boolean satisfiability is the problem of determining whether there exists an assignment of truth values to Boolean variables that makes a propositional formula true. A formula in conjunctive normal form (CNF) is a conjunction of clauses, where each clause is a disjunction of literals.
The movers problem is naturally modeled as a planning problem over discrete time. Instead of searching directly over sequences of actions, we encode the existence of a valid plan of length T as a SAT formula:

$$\Phi_T.$$

If Φ_T is satisfiable, then there exists a valid plan of length T. If it is unsatisfiable, no plan exists within that time horizon.
This approach is useful because modern SAT solvers can efficiently handle large structured Boolean formulas. The problem is solved by increasing the time horizon until the first satisfiable formula is found:

$$T = 0, 1, 2, \ldots$$

The first satisfiable T corresponds to an optimal plan with respect to the number of time steps.

4 Boolean Variable Representation

The encoding uses several families of propositional variables. The most important ones are shown in Table 1.

Variable	Meaning
`atFloor`(p, f, t)	Person p is on floor f at time t
`atFloor`(i, f, t)	Item i is on floor f at time t
`ascends`(p, t)	Person p ascends one floor during step t
`descends`(p, t)	Person p descends one floor during step t
`carries`(p, i, t)	Person p carries item i during step t

Table 1: Boolean variable families used in the SAT encoding.

In the implementation, each symbolic variable is assigned a unique positive integer using a PySAT `IDPool`. This is necessary because SAT solvers operate on integer variable identifiers rather than symbolic names. A separate mapping is produced so that the integer identifiers can be translated back into readable names such as:

```
atFloor(Alice, 2, 5)
```

or

```
carries(Bob, Sofa, 7).
```

For a problem with n floors, m movers, k items, and time horizon T, the variable families have the following approximate sizes:

- Person position variables: $m \cdot n \cdot (T + 1)$.
- Item position variables: $k \cdot n \cdot (T + 1)$.
- Ascend variables: $m \cdot T$.
- Descend variables: $m \cdot T$.
- Carry variables: $m \cdot k \cdot T$.

Therefore, Task 2 uses approximately

$$mn(T + 1) + kn(T + 1) + 2mT + mkT$$

Boolean variables.

5 Constraint Encoding

The SAT formula is built from multiple groups of constraints. Task 1 contains constraints for movement only, while Task 2 extends these constraints with item and carrying rules.

5.1 Common Cardinality Constraints

Several rules require exactly-one or at-most-one constraints. For example, a mover must be on exactly one floor at every time step. The implementation builds these constraints using simple CNF combinators.
Given a set of literals $L = \{l_1, l_2, \ldots, l_r\}$:

At least one.

$$l_1 \vee l_2 \vee \cdots \vee l_r.$$

At most one.

$$\bigwedge_{1 \le a < b \le r} (\neg l_a \vee \neg l_b).$$

Exactly one.

$$(l_1 \vee \cdots \vee l_r) \wedge \bigwedge_{1 \le a < b \le r} (\neg l_a \vee \neg l_b).$$

The pairwise at-most-one encoding is not the most compact possible encoding, but it is simple, readable, and effective for the sizes considered in this project.

5.2 Task 1: Movement-Only Encoding

Task 1 models the movement of persons independently from the items. All persons begin at floor 0, and each person must reach a specified target floor at time T.
Let:

$$P = \{0, \ldots, m-1\}, \qquad F = \{0, \ldots, n-1\}.$$

A. Initial State

Each person starts on floor 0:

$$\texttt{atFloor}(p, 0, 0) \qquad \forall p \in P.$$

The encoding also explicitly forbids the person from being on any other floor at time 0:

$$\neg\texttt{atFloor}(p, f, 0) \qquad \forall p \in P,\ \forall f \in F,\ f \neq 0.$$

B. Goal State

For each person p, a target floor g_p is specified:

$$\texttt{atFloor}(p, g_p, T).$$

In Task 2, all persons must end at the ground floor, so $g_p = 0$ for every person.

C. Unique Floor

Each person must be on exactly one floor at every time step:

$$\text{ExactlyOne}\left(\texttt{atFloor}(p, 0, t), \ldots, \texttt{atFloor}(p, n-1, t)\right)$$

for every $p \in P$ and $t \in \{0, \ldots, T\}$.

D. Action Mutex

A person cannot ascend and descend at the same time:

$$\neg\texttt{ascends}(p,t) \vee \neg\texttt{descends}(p,t)$$

for every $p \in P$ and $t < T$.

E. Movement Preconditions

A person cannot ascend from the top floor:

$$\neg\texttt{ascends}(p,t) \vee \neg\texttt{atFloor}(p,n-1,t).$$

Similarly, a person cannot descend from the ground floor:

$$\neg\texttt{descends}(p,t) \vee \neg\texttt{atFloor}(p,0,t).$$

F. Effects and Frame Axioms

If a person ascends from floor $f-1$, then they must be on floor f at the next time step:

$$\texttt{atFloor}(p,f-1,t) \wedge \texttt{ascends}(p,t) \Rightarrow \texttt{atFloor}(p,f,t+1).$$

In CNF:

$$\neg\texttt{atFloor}(p,f-1,t) \vee \neg\texttt{ascends}(p,t) \vee \texttt{atFloor}(p,f,t+1).$$

If a person descends from floor $f+1$, then they must be on floor f at the next time step:

$$\texttt{atFloor}(p,f+1,t) \wedge \texttt{descends}(p,t) \Rightarrow \texttt{atFloor}(p,f,t+1).$$

In CNF:

$$\neg\texttt{atFloor}(p,f+1,t) \vee \neg\texttt{descends}(p,t) \vee \texttt{atFloor}(p,f,t+1).$$

Finally, if a person is idle, their floor is preserved:

$$\texttt{atFloor}(p,f,t) \wedge \neg\texttt{ascends}(p,t) \wedge \neg\texttt{descends}(p,t) \Rightarrow \texttt{atFloor}(p,f,t+1).$$

In CNF:

$$\neg\texttt{atFloor}(p,f,t) \vee \texttt{ascends}(p,t) \vee \texttt{descends}(p,t) \vee \texttt{atFloor}(p,f,t+1).$$

5.3 Task 2: Movement and Carrying Encoding

Task 2 extends Task 1 by introducing items and carrying constraints. All persons and all items must end on floor 0. An item can only move downward when carried by a descending person. Let I be the set of items.

G. Item Initial State

Each item starts at its specified initial floor:

$$\texttt{atFloor}(i,f_i,0) \qquad \forall i \in I.$$

The encoding also forbids the item from being on any other floor at time 0:

$$\neg\texttt{atFloor}(i,f,0) \qquad \forall i \in I,\ \forall f \neq f_i.$$

H. Item Goal State

Every item must be on the ground floor at time T:

$$\texttt{atFloor}(i, 0, T) \qquad \forall i \in I.$$

I. Item Unique Floor

Each item must be on exactly one floor at every time step:

$$\text{ExactlyOne}\left(\texttt{atFloor}(i, 0, t), \ldots, \texttt{atFloor}(i, n-1, t)\right)$$

for every item i and time t.

J. Carrying Preconditions

If a person carries an item, the person and item must be co-located. The implementation encodes this by requiring that if person p carries item i and person p is on floor f, then item i is also on floor f:

$$\texttt{carries}(p, i, t) \wedge \texttt{atFloor}(p, f, t) \Rightarrow \texttt{atFloor}(i, f, t).$$

In CNF:

$$\neg\texttt{carries}(p, i, t) \vee \neg\texttt{atFloor}(p, f, t) \vee \texttt{atFloor}(i, f, t).$$

Carrying also implies descending:

$$\texttt{carries}(p, i, t) \Rightarrow \texttt{descends}(p, t),$$

which becomes:

$$\neg\texttt{carries}(p, i, t) \vee \texttt{descends}(p, t).$$

K. Carrying Mutex

Each person can carry at most one item at a time:

$$\text{AtMostOne}\left(\texttt{carries}(p, i_1, t), \ldots, \texttt{carries}(p, i_k, t)\right).$$

Each item can be carried by at most one person at a time:

$$\text{AtMostOne}\left(\texttt{carries}(p_1, i, t), \ldots, \texttt{carries}(p_m, i, t)\right).$$

L. Ascending Precludes Carrying

A person cannot carry an item while ascending:

$$\neg\texttt{ascends}(p, t) \vee \neg\texttt{carries}(p, i, t).$$

This is consistent with the rule that items can only be moved downward.

M. Item Successor-State Axioms

If an item is on floor $f+1$ and is carried by a person at time t, then it moves down to floor f at time $t+1$:

$$\texttt{atFloor}(i, f+1, t) \land \texttt{carries}(p, i, t) \Rightarrow \texttt{atFloor}(i, f, t+1).$$

In CNF:

$$\neg\texttt{atFloor}(i, f+1, t) \lor \neg\texttt{carries}(p, i, t) \lor \texttt{atFloor}(i, f, t+1).$$

If an item is not carried, it remains on the same floor:

$$\texttt{atFloor}(i, f, t) \land \neg\texttt{carries}(p_1, i, t) \land \cdots \land \neg\texttt{carries}(p_m, i, t) \Rightarrow \texttt{atFloor}(i, f, t+1).$$

The corresponding CNF clause is:

$$\neg\texttt{atFloor}(i, f, t) \lor \texttt{carries}(p_1, i, t) \lor \cdots \lor \texttt{carries}(p_m, i, t) \lor \texttt{atFloor}(i, f, t+1).$$

6 Solver Execution and Model Decoding

The solver supports two execution modes.

6.1 Fixed-Horizon Solving

In fixed-horizon mode, the user specifies an exact time horizon T. The backend generates the CNF formula Φ_T and passes it to Glucose3. If the formula is satisfiable, the backend returns a decoded plan. If the formula is unsatisfiable, the backend reports that no plan exists for exactly that number of steps.

6.2 Optimal Solving by Iterative Deepening

In optimal mode, the backend tries increasing values of T:

$$0, 1, 2, \ldots, T_{\max}.$$

For each value, it generates and solves a new CNF formula. The first satisfiable result is returned. This guarantees that the returned plan is optimal with respect to the number of time steps, assuming the search reaches a satisfiable horizon.

Algorithm 1 Iterative deepening strategy for finding the shortest plan.

```
for T = 0 to T_max do
    Encode problem at horizon T
    Run SAT solver
    if SAT then
        Decode model and return plan
    end if
end for
return UNSAT within bound
```

6.3 Model Decoding

The SAT solver returns a model as a list of signed integers. Positive integers represent variables assigned true, while negative integers represent variables assigned false. The decoder extracts the true variables and reconstructs:

- the position of every mover at each time step,
- the position of every item at each time step,
- each mover's action during each transition.

The decoded output is represented as a list of time-step objects. Each time step contains the current positions and the actions performed from that time step to the next one.

7 DIMACS and Variable Mapping Output

The system exports two important files for verification and debugging.

7.1 DIMACS CNF

The generated SAT instance is exported in DIMACS CNF format. The file contains:

- comment lines beginning with `c`,
- a problem line of the form `p cnf <variables> <clauses>`,
- one line per clause, ending in `0`.

An example clause line may look like:

```
-10 -274 26 0
```

This allows the generated formula to be inspected manually or passed to an external SAT solver.

7.2 Variable Mapping

The variable mapping translates integer SAT variable identifiers into readable names. For example:

```
{
  "274": "ascends(Alice, 0)",
  "275": "descends(Alice, 0)",
  "710": "carries(Alice, Sofa, 0)"
}
```

This mapping is essential for interpreting a satisfying truth assignment.

8 Frontend Functionality

The frontend makes the SAT solver accessible through an interactive interface. The user can configure the problem without manually writing SAT clauses or JSON requests.

8.1 Problem Setup Panel

The problem setup panel allows the user to select:

- the task type,
- the total number of floors,
- the number of movers,
- item names and initial item floors,
- custom mover names,
- target floors for Task 1,
- solving mode,
- maximum number of steps or exact time horizon.

For Task 1, advanced options allow the user to set target floors for each mover. For Task 2, the target is fixed: every mover and every item must finish on floor 0.

8.2 Results Panel

After the solver runs, the results panel displays whether the instance is satisfiable or unsatisfiable. If a solution exists, the panel shows:

- the solve ID,
- the solver runtime,
- the optimal number of steps,
- buttons for downloading the DIMACS CNF and mapping,
- playback controls for the solution timeline.

8.3 Building Visualizer

The building visualizer represents the floors graphically. Movers are shown as person icons, while items are shown as boxes. At each time step, the visualizer updates the positions of all persons and items according to the decoded SAT model.

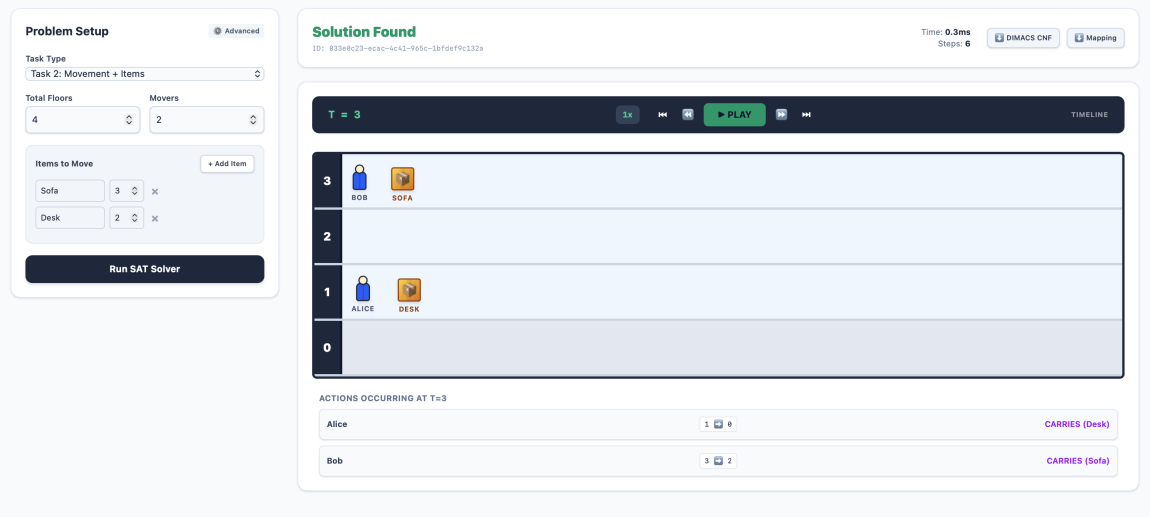


Figure 2: Frontend visualization of a satisfiable Task 2 instance with two movers and two items.

8.4 Action Log

The action log describes what happens at the selected time step. For example, it may show that Alice descends from floor 2 to floor 1, or that Bob carries a table from floor 1 to floor 0.

9 Complexity and Scalability Analysis

9.1 Variable Growth

Let:

$$n = \text{number of floors},$$
$$m = \text{number of movers},$$
$$k = \text{number of items},$$
$$T = \text{time horizon}.$$

For Task 1, the number of variables is approximately:

$$mn(T+1) + 2mT.$$

The first term represents mover positions, while the second term represents the ascend and descend action variables.
For Task 2, item positions and carrying variables are added:

$$mn(T+1) + kn(T+1) + 2mT + mkT.$$

Thus, the number of variables grows linearly with the time horizon T, but also depends on the product mk because every mover-item pair may correspond to a carrying variable at every step.

9.2 Clause Growth

The largest source of clauses comes from exactly-one and at-most-one constraints. Because the implementation uses pairwise at-most-one clauses, enforcing uniqueness over r literals requires:

$$\binom{r}{2}$$

binary clauses.
For example, requiring a person to be on exactly one of n floors at a given time step requires:

$$1 + \binom{n}{2}$$

clauses. This is repeated for every person and every time step.
Task 2 adds similar constraints for items, as well as carrying mutex constraints between movers and items. Therefore, the number of clauses grows polynomially in the input parameters for a fixed horizon.

9.3 SAT Solving Complexity

Although the encoding size is polynomial in n, m, k, and T, SAT solving is NP-complete in the worst case. Therefore, there is no guarantee that every large instance will be solved quickly.
In practice, modern SAT solvers such as Glucose3 exploit structure, unit propagation, clause learning, and branching heuristics. For the tested problem sizes, the solver usually returns results within milliseconds.

9.4 Practical Scalability

The most important factor affecting performance is the chosen time horizon. In fixed-horizon mode, an unrealistically small T often produces an unsatisfiable instance quickly, while a larger T may produce many more variables and clauses. In iterative deepening mode, the backend solves a sequence of formulas, which is useful for optimality but can become expensive when the optimal plan requires many steps.

10 Experimental Evaluation

This section describes the main test cases used to evaluate the solver.

10.1 Case 1: Task 1 Movement with Different Goal Floors

For the movement-only task, three movers start on floor 0:

$$\texttt{atFloor(Alice,0,0)}, \quad \texttt{atFloor(Bob,0,0)}, \quad \texttt{atFloor(Eve,0,0)}.$$

In the tested configuration, the building has seven floors and the target floors are:

$$\texttt{Alice}@6, \qquad \texttt{Bob}@4, \qquad \texttt{Eve}@2.$$

The solver finds an optimal plan in six steps. This is expected because Alice must ascend from floor 0 to floor 6, which requires six upward movements. Bob and Eve reach their target floors earlier and then remain idle while Alice completes the longest path.

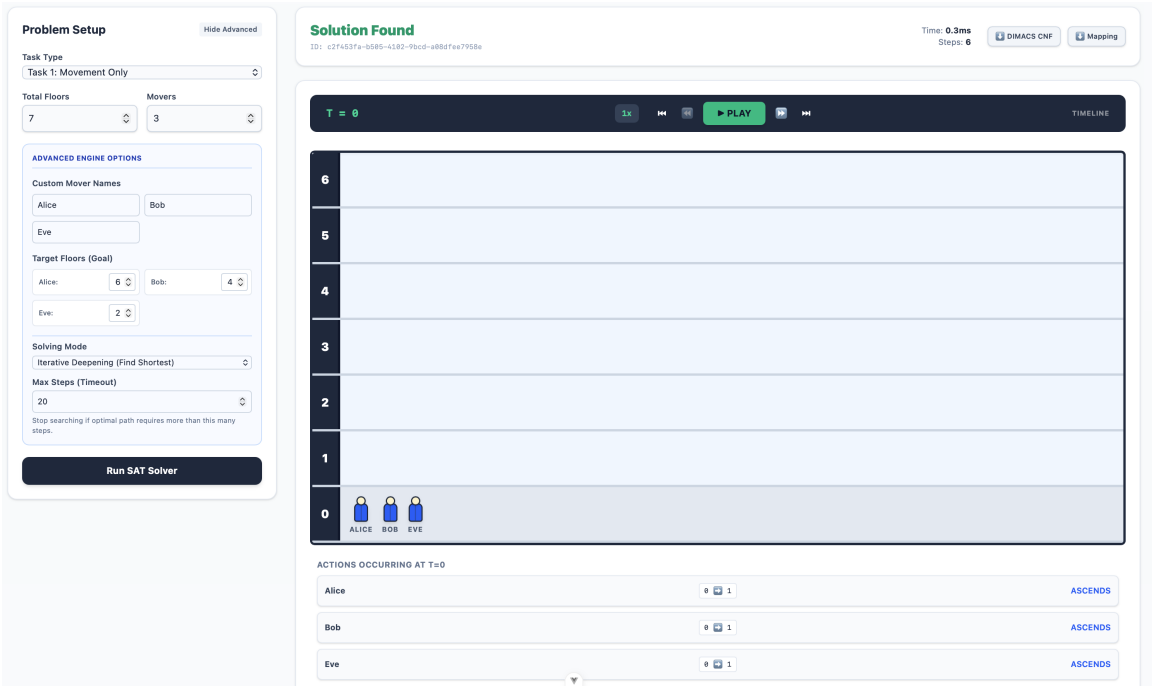


Figure 3: Initial state of a Task 1 instance: all movers start on the ground floor.

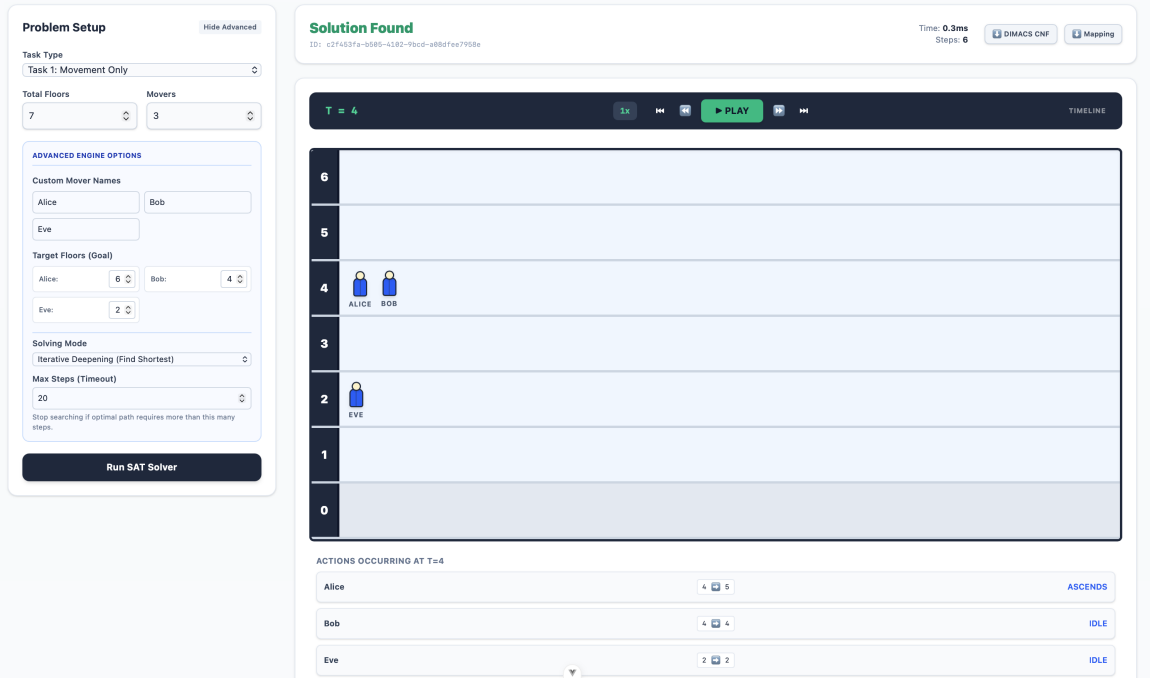


Figure 4: Intermediate state of the Task 1 solution, where some movers have already reached their target floors.

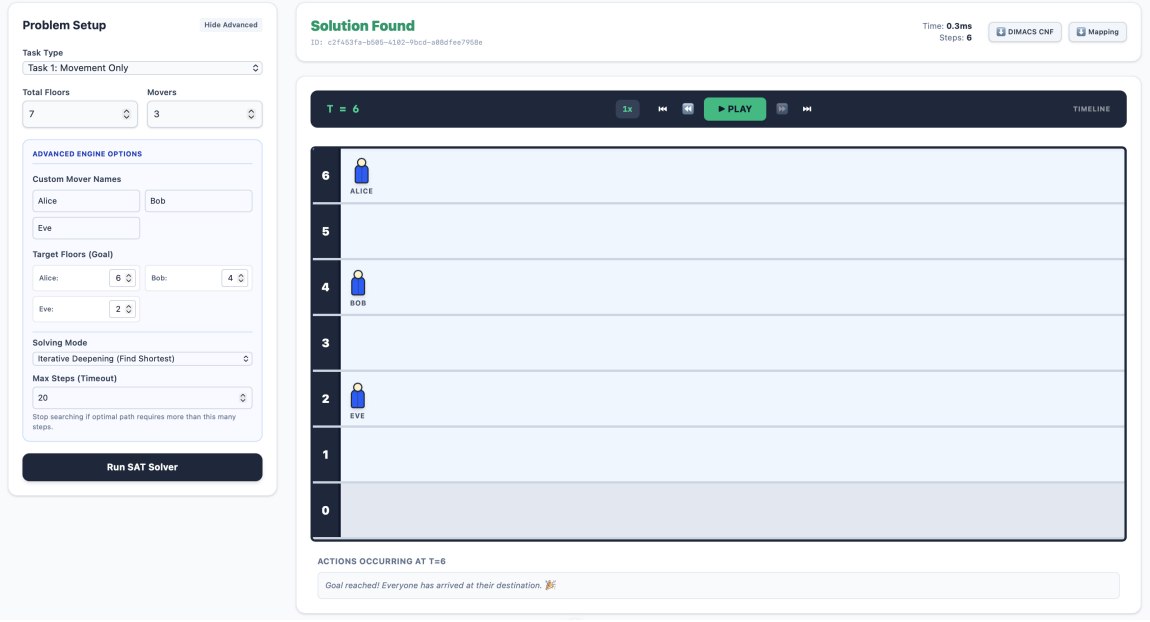


Figure 5: Final state of the Task 1 solution: Alice, Bob, and Eve are on their required target floors.

10.2 Case 2: Task 1 Unsatisfiable Within a Limited Horizon

The same Task 1 goal cannot be reached if the maximum allowed time horizon is too small. For example, Alice must reach floor 6 from floor 0, so any solution requires at least six time steps. If the search is stopped at $T_{\max} = 4$, the solver correctly reports the instance as unsatisfiable within the given bound.

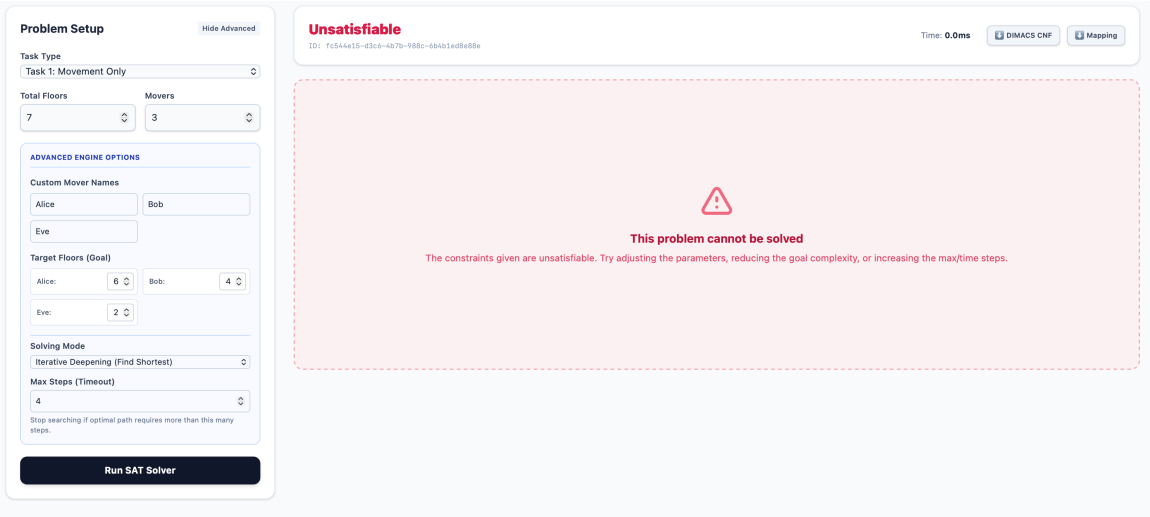


Figure 6: Task 1 unsatisfiable result when the maximum search horizon is too small to reach the target floors.

10.3 Case 3: Trivial Task 1 Goal

If the target floor is already floor 0, then the initial state satisfies the goal. The solver correctly returns a satisfiable result at $T = 0$.

10.4 Case 4: Task 2 with Three Items

For the carrying task, a tested configuration uses three movers and three items:

$$\texttt{flowerpot}@2, \qquad \texttt{chair}@1, \qquad \texttt{table}@2.$$

The solver finds a valid plan in four steps. The final state places all movers and all items on floor 0.

10.5 Case 5: Task 2 Unsatisfiable Fixed-Horizon Instance

In this frontend test, the building has seven floors, two movers, and three items:

$$\texttt{Sofa}@3, \qquad \texttt{Desk}@4, \qquad \texttt{Chair}@6.$$

The solver is executed in fixed-horizon mode with an exact time horizon of $T = 10$. The instance is reported as unsatisfiable, showing that the system correctly detects when no valid plan exists within the selected number of time steps. This also confirms that the frontend properly displays bounded UNSAT results returned by the backend.

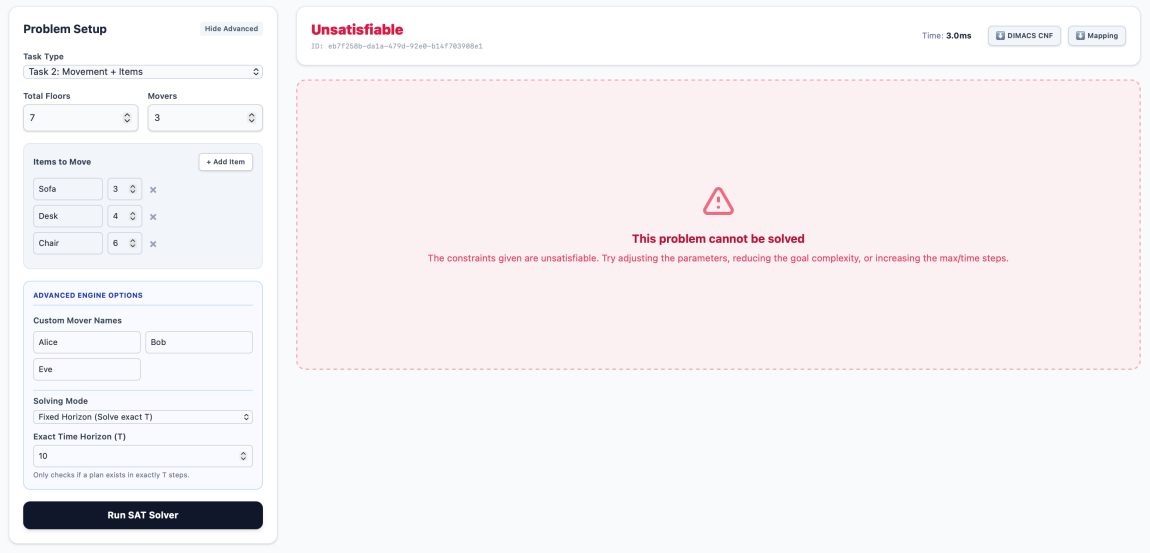


Figure 7: Frontend output for an unsatisfiable fixed-horizon Task 2 instance.

10.6 Case 6: Full Example with Five Items

The full example configuration contains:

$$\texttt{flowerpot@2}, \quad \texttt{chair@1}, \quad \texttt{table@2}, \quad \texttt{piano@1}, \quad \texttt{sofa@2}.$$

The solver finds an optimal solution in six steps for this full configuration. This confirms that the Task 2 encoding scales beyond the minimal three-item example.

10.7 Case 7: Larger Custom Frontend Instance

A larger frontend example uses seven floors, three movers, and four items:

$$\texttt{Sofa@3}, \quad \texttt{Desk@4}, \quad \texttt{Chair@6}, \quad \texttt{Table@1}.$$

The frontend visualization shows the movers and items progressing through the building until every object reaches floor 0. The solution is displayed with a timeline, action log, and final confirmation message.

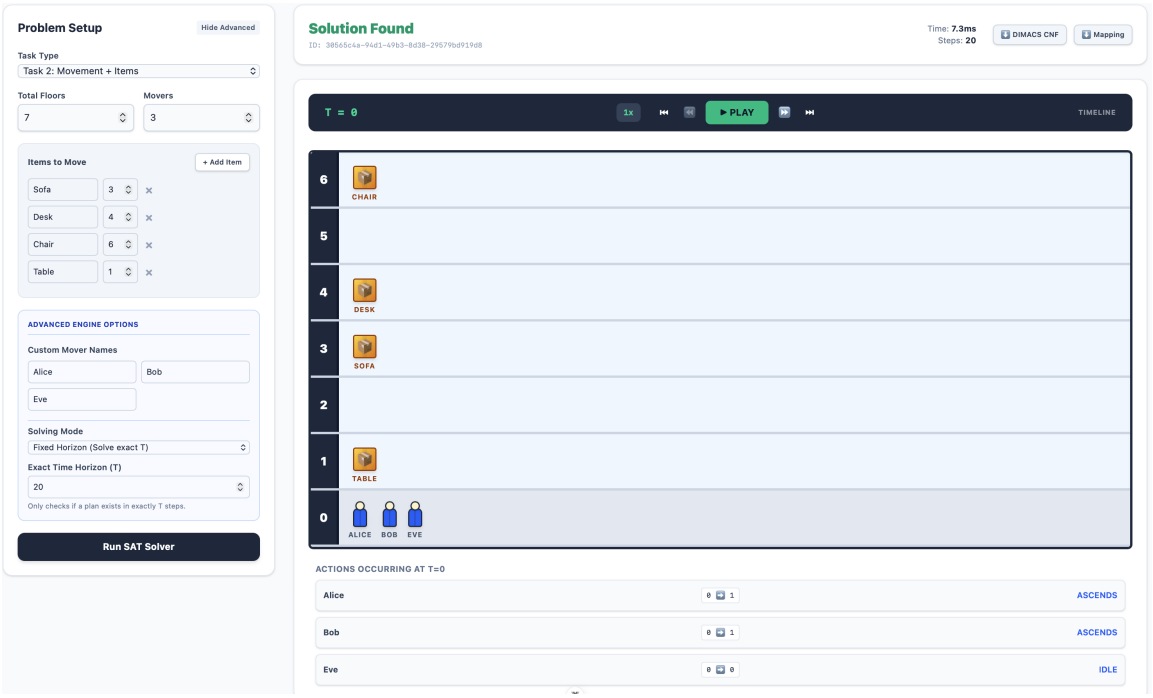


Figure 8: Initial state of a larger Task 2 instance with seven floors, three movers, and four items.

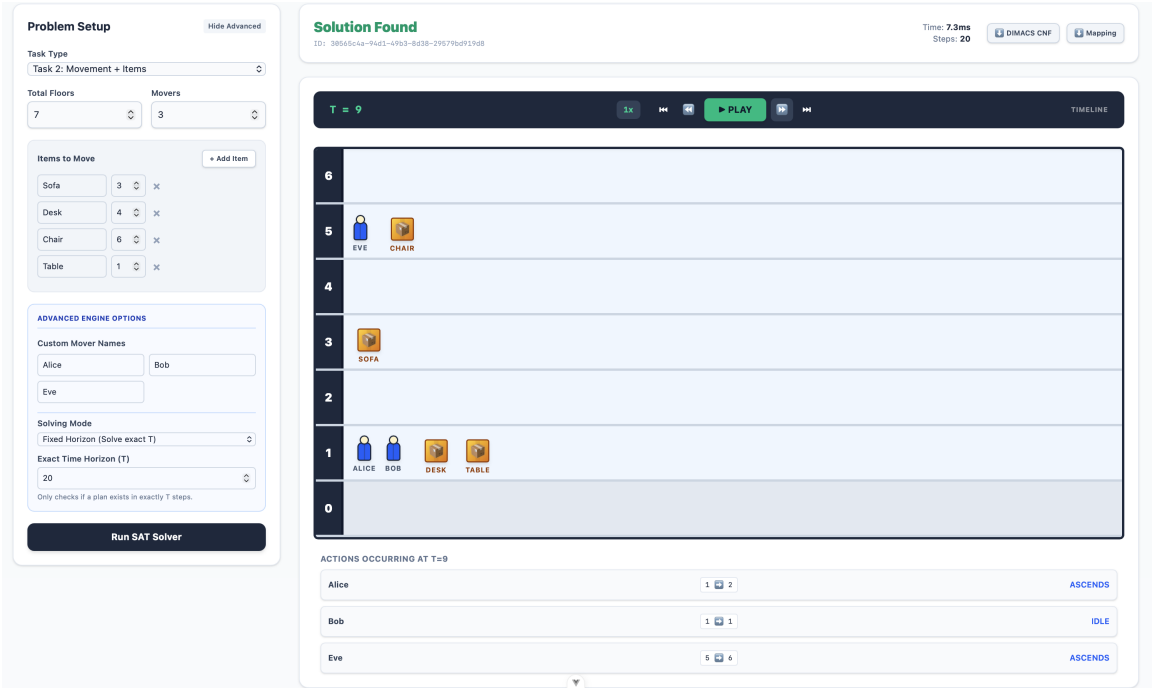


Figure 9: Intermediate state of the larger instance, showing movers carrying items during the plan.

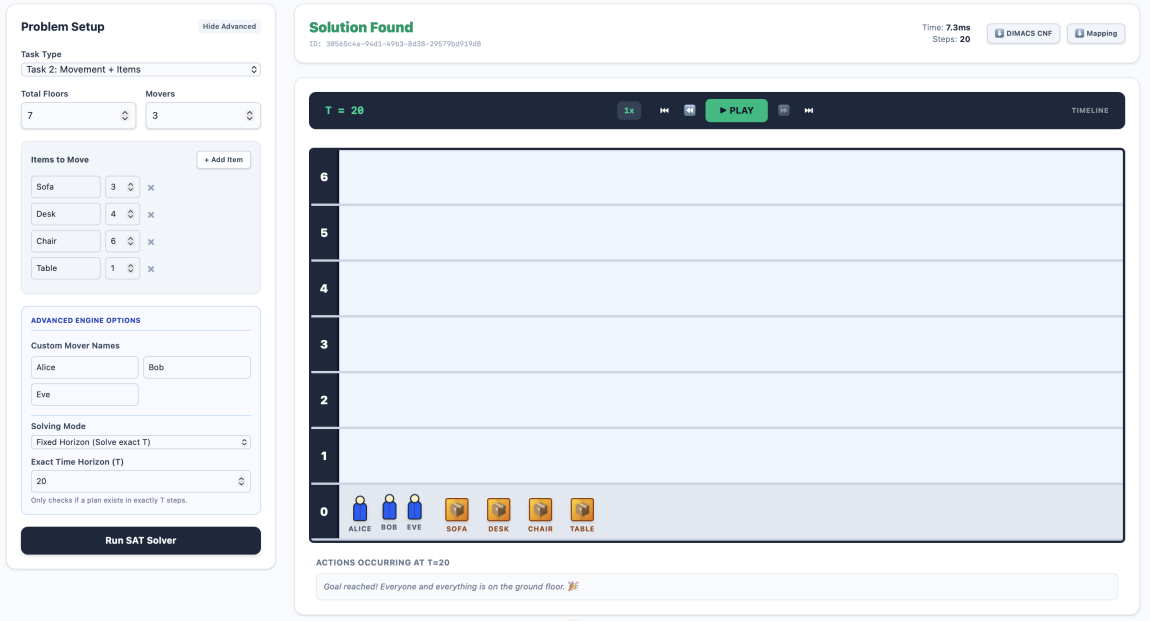


Figure 10: Final state of the larger instance, where all movers and items have reached the ground floor.

11 Design Decisions and Issues

11.1 General CNF Instead of 3-CNF

The encoding uses general CNF clauses rather than forcing every clause to contain exactly three literals. Although 3-SAT is important theoretically, converting every constraint into 3-CNF would require additional auxiliary variables and would make the implementation less readable. General CNF is accepted directly by the PySAT solver interface, so this choice keeps the encoding simpler.

11.2 Pairwise Cardinality Encoding

The system uses pairwise clauses for at-most-one constraints. This produces $O(r^2)$ clauses for a group of r literals, but the implementation is straightforward and works well for the tested problem sizes.

11.3 Iterative Deepening for Optimality

Finding the shortest plan is achieved by repeatedly solving fixed-horizon instances. This approach is easy to implement and gives a strong optimality guarantee: the first satisfiable horizon is the shortest possible one.

11.4 Readable Mapping Output

The project explicitly exports a mapping from integer SAT variables to readable names. This is useful both for debugging and for satisfying the requirement that the SAT instance and truth assignment be interpretable.

11.5 Frontend Visualization

The frontend is not required for the logical encoding itself, but it makes the solver much easier to understand. Instead of only returning a list of Boolean assignments, the system shows a step-by-step animation of the plan.

12 Limitations

The current implementation has some limitations:

- Large instances can generate many clauses, especially due to pairwise at-most-one constraints.
- Iterative deepening may require solving many formulas before finding the optimal horizon.
- The in-memory cache used for retrieving CNF and mapping files does not persist after the backend restarts.
- The frontend is designed for moderate-sized examples; very large buildings or many movers/items may become visually crowded.
- The carrying model assumes that items only move down while carried. It does not model more complex actions such as passing items between people or carrying while staying on the same floor.

13 Conclusion

This project demonstrates how a logistics planning problem can be reduced to Boolean satisfiability. By introducing variables for positions, actions, and carrying relations, the movement of movers and items through a building can be represented as a CNF formula.
The implementation supports both a movement-only version and a full movement-plus-items version of the problem. The backend uses PySAT and Glucose3 to solve the generated formulas, while the frontend provides an interactive way to configure and visualize problem instances.
The solver successfully handles satisfiable and unsatisfiable cases, exports DIMACS CNF and variable mappings, and finds optimal plans using iterative deepening. Overall, the project connects theoretical concepts from SAT and computational logic with a practical planning application.

Appendices

A User Guide

The project can be executed either with Docker Compose or manually by starting the backend and frontend separately. The recommended method is Docker Compose, because it starts both services with a single command and avoids dependency conflicts.

A.1 Docker Setup

The repository includes a `docker-compose.yml` file at the project root. It defines two services:

- `backend`: builds the FastAPI backend from `./backend` and exposes it on port `8000`.
- `frontend`: builds the Vue/Vite frontend from `./frontend` and exposes it on port `3000`.

To start the complete application using Docker, run the following command from the root directory of the project:

```
docker-compose up --build
```

After the containers are running, the application can be accessed at:

```
Frontend UI:      http://localhost:3000
Backend API docs: http://localhost:8000/docs
Backend API:      http://localhost:8000/api
```

To stop the application, press:

```
Ctrl+C
```

Alternatively, to stop and remove the containers, run:

```
docker-compose down
```

The frontend container is configured with the environment variable:

```
VITE_API_BASE_URL=http://localhost:8000/api
```

This means that, when the frontend is opened in the browser at `localhost:3000`, API requests are sent to the backend at `localhost:8000/api`.

A.2 Backend Setup Without Docker

To run the backend manually:

```
cd backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

On Windows, the virtual environment can be activated with:

```
.venv\Scripts\activate
```

The backend runs at:

```
http://localhost:8000
```

The API documentation is available at:

```
http://localhost:8000/docs
```

A.3 Frontend Setup Without Docker

To run the frontend manually:

```
cd frontend
npm install
npm run dev
```

When running through Docker, the frontend is available at:

```
http://localhost:3000
```

A.4 CLI Examples

Task 1 example:

```
python cli.py --floors 3 --persons 3 --task 1 --goal-floor 2
```

Task 2 example:

```
python cli.py --floors 3 --persons 2 --items chair:1 table:2
```

Fixed-horizon solving:

```
python cli.py --floors 3 --persons 2 --items chair:1 table:2 --fixed-t 5
```

Exporting DIMACS CNF:

```
python cli.py --floors 3 --persons 2 --items chair:1 table:2 --cnf-out problem.cnf
```

B API Endpoints

Method	Path	Description
POST	`/api/solve`	Find the optimal solution using iterative deepening
POST	`/api/solve/fixed`	Solve for an exact time horizon
GET	`/api/cnf/{solve_id}`	Retrieve the DIMACS CNF of a previous solve
GET	`/api/mapping/{solve_id}`	Retrieve the variable mapping of a previous solve
GET	`/api/health`	Health check endpoint

Table 2: REST API endpoints exposed by the backend.

C Example API Request

```
curl -X POST http://localhost:8000/api/solve \
  -H "Content-Type: application/json" \
  -d `{
    "task": 2,
    "num_floors": 3,
    "num_persons": 2,
    "person_names": ["Alice", "Bob"],
    "items": [
      {"name": "Sofa", "initial_floor": 2},
      {"name": "Desk", "initial_floor": 1}
    ],
    "max_steps": 20
  }`
```

D File Structure

```
TOC_MOVERS/
|-- docker-compose.yml
|-- README.md
|-- backend/
|   |-- Dockerfile
|   |-- README.md
|   |-- requirements.txt
|   |-- cli.py
|   |-- app/
|   |   |-- config.py
|   |   |-- main.py
|   |   |-- api/
|   |   |   |-- routes.py
|   |   |-- encoding/
|   |   |   |-- common_clauses.py
|   |   |   |-- task1_encoder.py
|   |   |   |-- task2_encoder.py
|   |   |   |-- variables.py
|   |   |-- models/
|   |   |   |-- problem.py
|   |   |-- output/
|   |   |   |-- dimacs.py
|   |   |   |-- mapping.py
|   |   |-- solver/
|   |       |-- decoder.py
|   |       |-- runner.py
|-- frontend/
|   |-- Dockerfile
|   |-- index.html
|   |-- public/
|   |-- src/
|       |-- main.js
|       |-- App.vue
|       |-- api/
|       |   |-- client.js
|       |-- assets/
|       |   |-- main.css
|       |-- components/
|           |-- ActionLog.vue
|           |-- BuildingVisualizer.vue
|           |-- ProblemSetup.vue
|           |-- ResultsPanel.vue
```